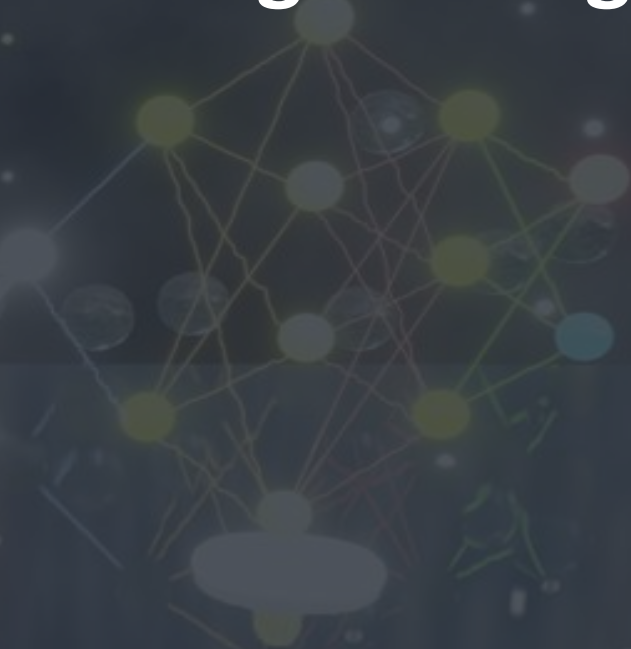
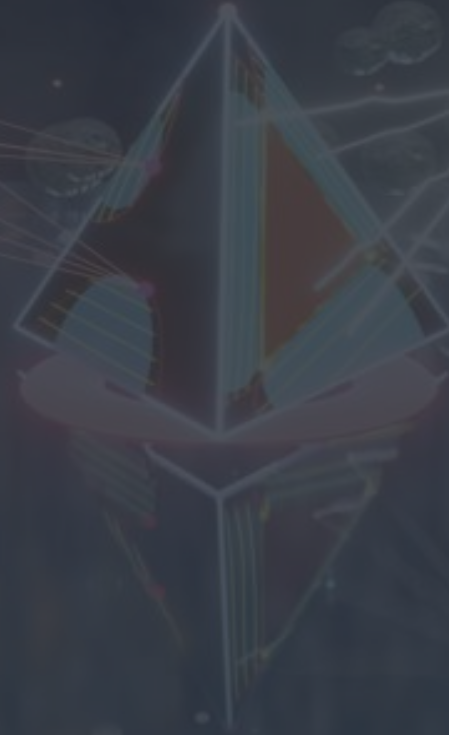
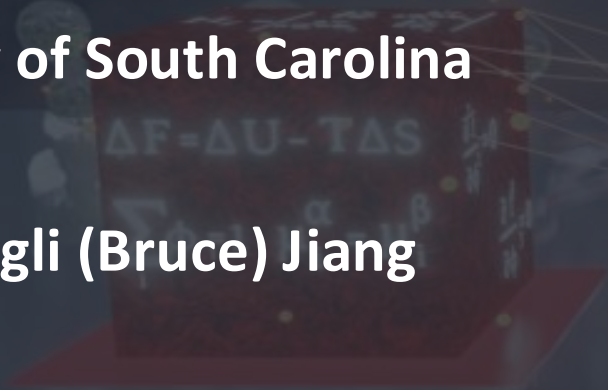


ECHE 589 Machine Learning in Chemical Engineering

Fall 2026

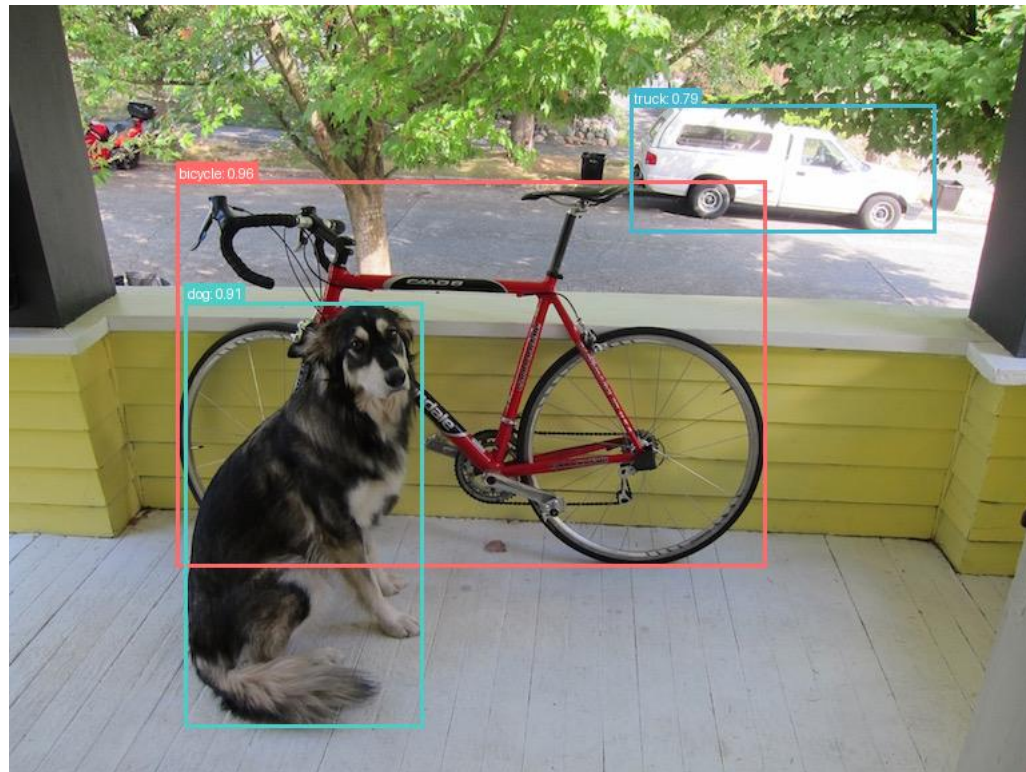
University of South Carolina

Prof. Shengli (Bruce) Jiang



What Can Machine Learning Do?

<https://cnn-playground.live/vision>



<https://kaushallab.mtu.edu/fragberta/>

SMILES REGISTRY MANUAL E... 1/1 IN: 1 VERIFIED: 1 FAILED: 0

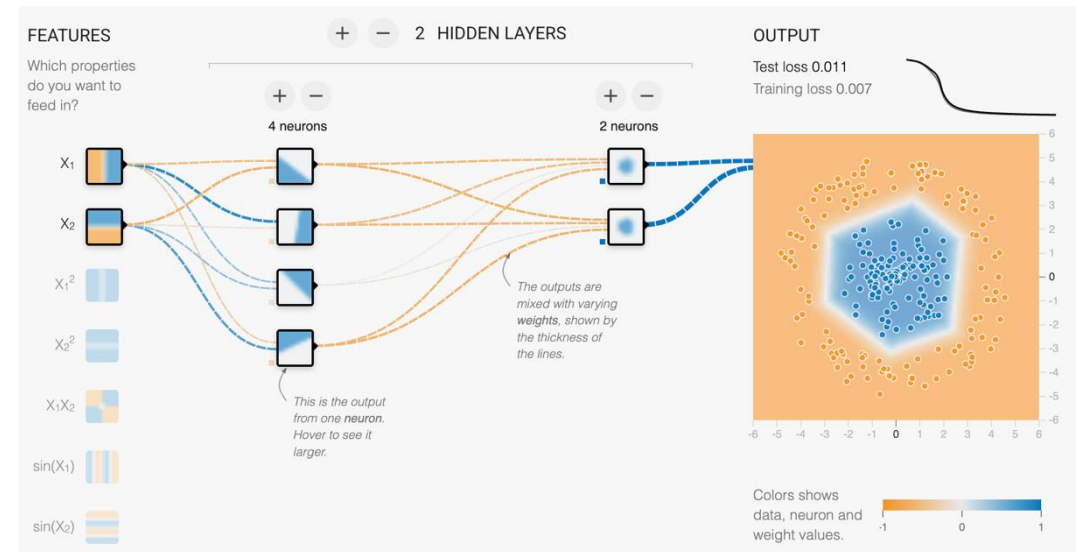
PREDICTION RESULTS FREESOLV Manual Entry

OS 1 PREDICTIONS CSV

CCO

HO CCO -5.643938

<https://playground.tensorflow.org/>

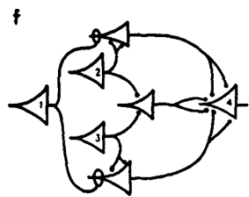


Machine Learning Is Older Than You Think

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH and WALTER H. PITTS

Bulletin of Mathematical Biophysics, Vol. 5, 1943, p. 115-133

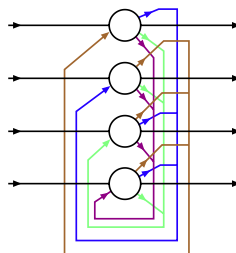


McCulloch and Pitts publish a paper proposing a mathematical representation of nervous activity based on propositional logic, introducing an early model of a neural network.

1943

John Hopfield introduces a recurrent neural network that stores and retrieves patterns as associative memories.

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j + \sum_i \theta_i s_i$$



1982

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

1986

Transformer

2017



Dr. Jiang studied ML 2018

you arrive in ECHE 589

2026

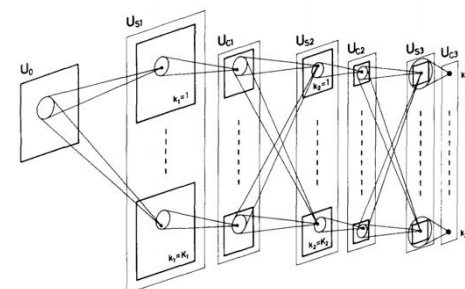
Explosion of interest and applications by BigTech

2012

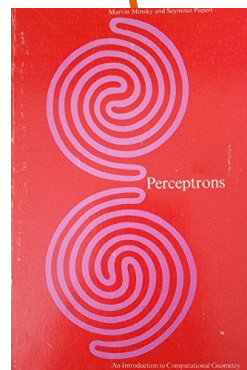


AlexNet reduces ImageNet top-5 error to 15.3%, compared with 26.2% for the second-place system, using **1.2 million images** and **two GPUs**.

Kunihiko Fukushima introduces the Neocognitron, a multilayer network that recognizes patterns despite shifts in position and helps lay the foundation for modern CNNs.



Minsky and Papert identify important limitations of single-layer perceptrons: linear decision boundary.



1969

Frank Rosenblatt introduces the perceptron, an early learning algorithm for image recognition.

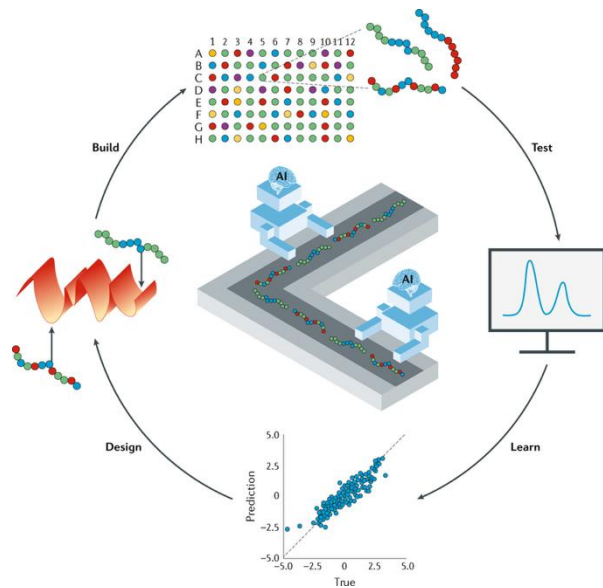


1957

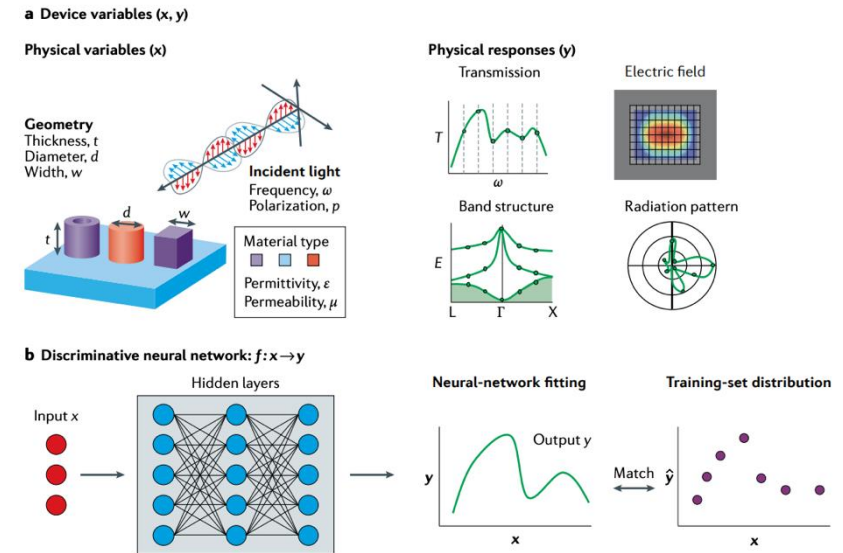
Big Bang



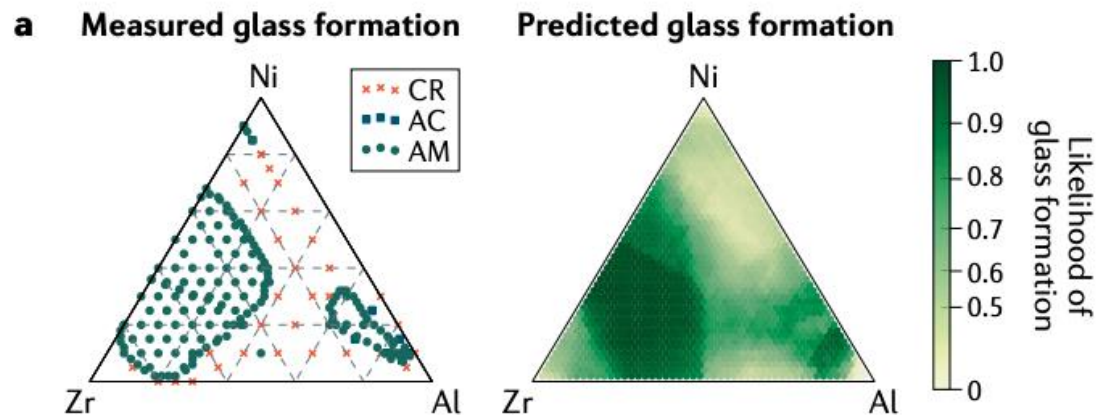
Machine Learning Accelerates Materials Discovery and Design



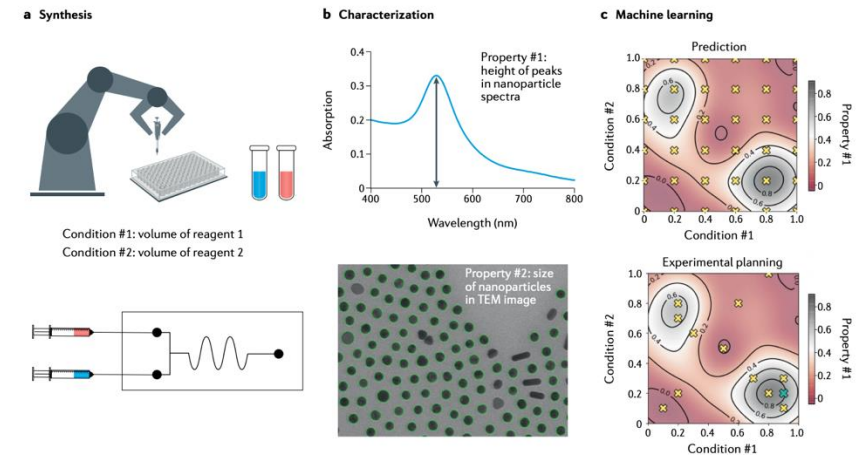
*“Machine Learning in combinatorial **polymer chemistry**”*
Gormley and Webb



*“Deep neural networks for the evaluation and design of **photonic devices**”*
Jiang, Chen, and Fan



*“Machine learning for **alloys**”*
Hart, Mueller, Toher, Curtarolo



*“**Nanoparticle synthesis** assisted by machine learning”*
Tao, Wu, Aldeghi, Aspuru-Guzik, and Kumacheva

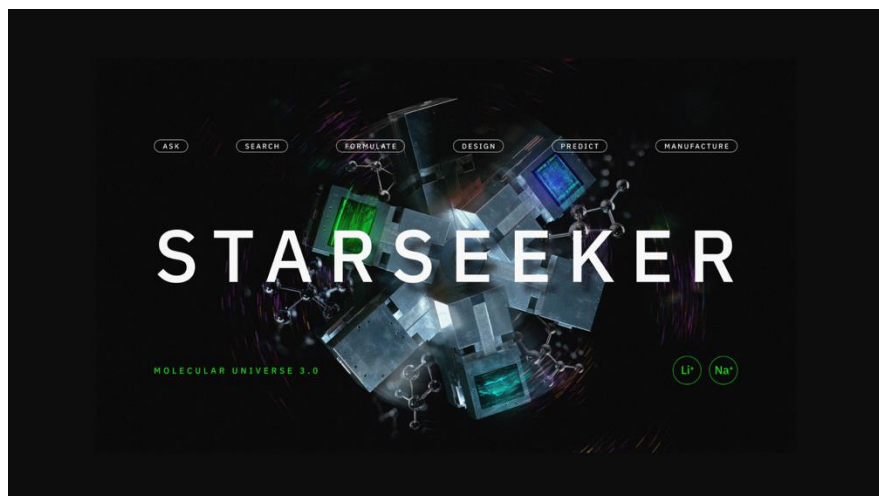
Machine Learning Is Already Used in the Chemical Industry



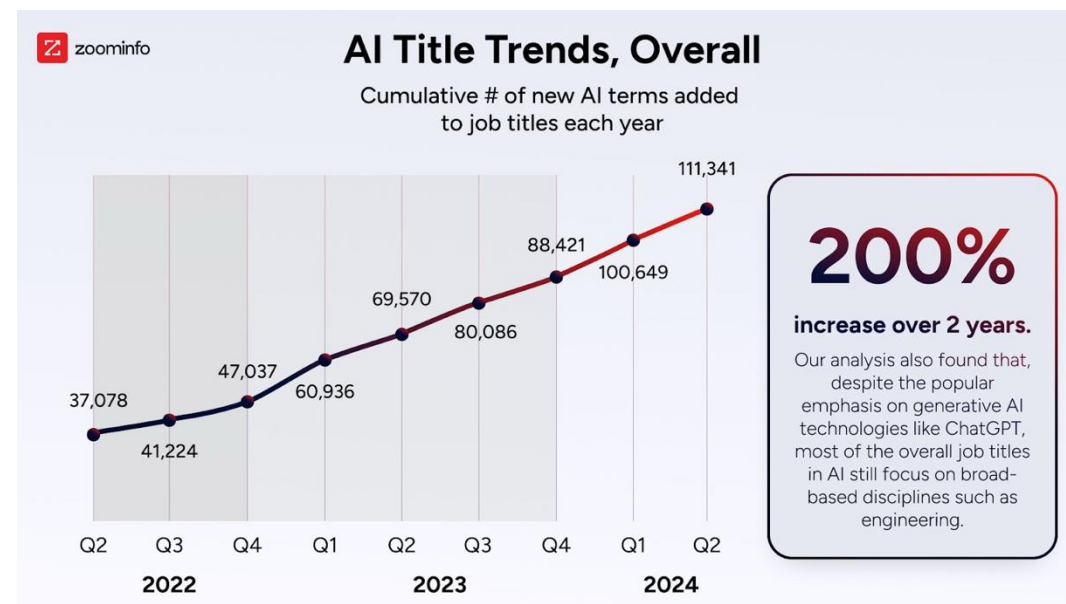
Detect equipment failure before production stops.



Predict reaction performance and recommend operating conditions for styrene production.

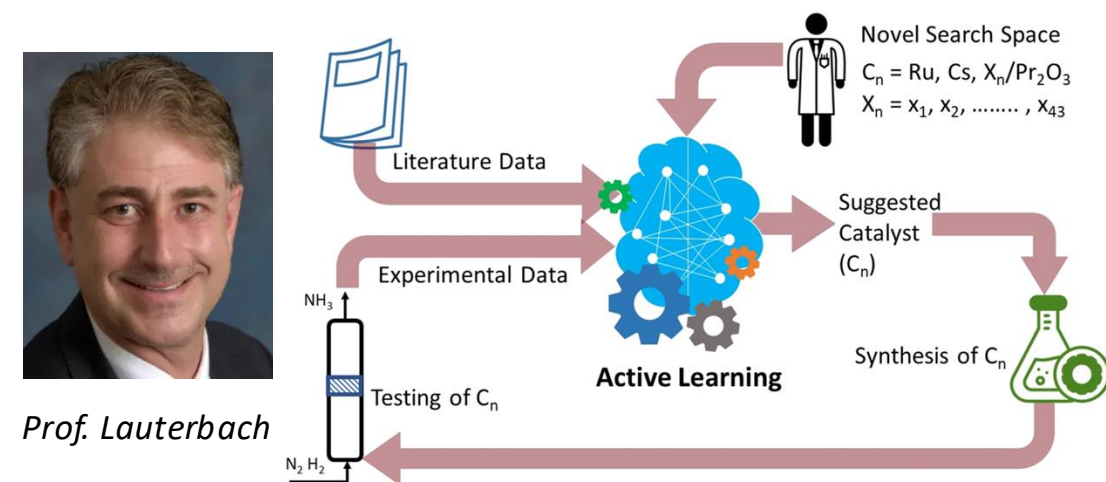


Predict promising battery materials before costly synthesis and testing.



Machine Learning Across South Carolina Engineering

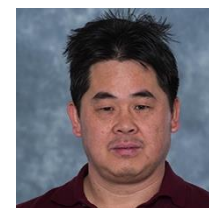
ML Discovers Ammonia Catalysts



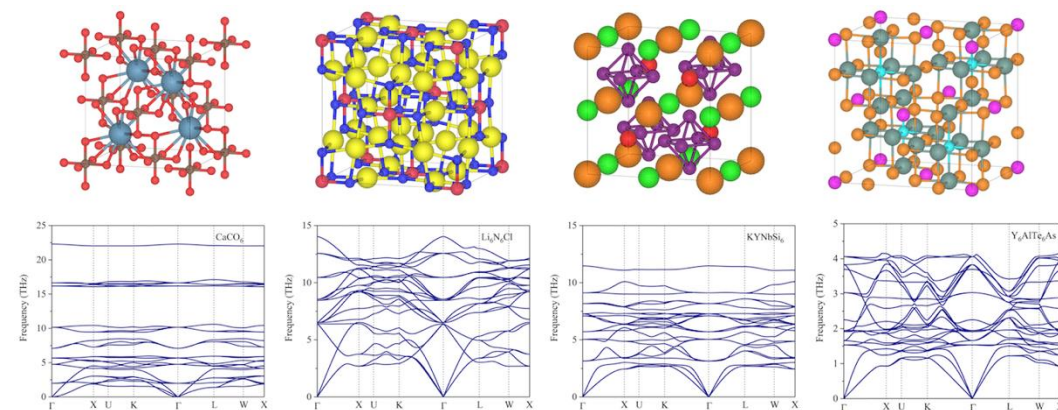
AI Screens Nearly 800,000 Crystals



Prof. Ming Hu



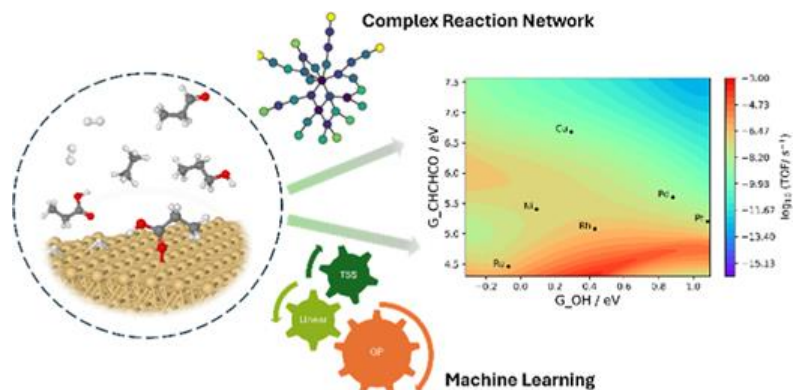
Prof. Jianjun Hu



ML Accelerates First-Principles Study

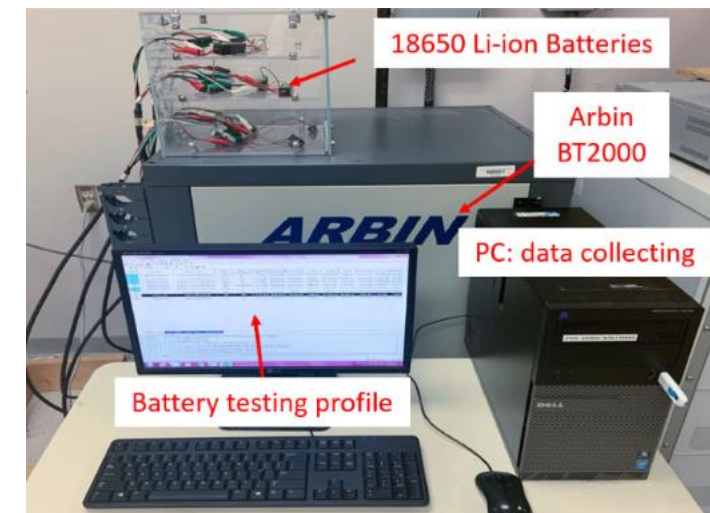


Prof. Heyden



Prof. Zhang

ML for Lithium-Ion Battery Diagnosis and Prognosis



Course Roadmap

Classes	Topics	Guiding Questions
Week 01 & 02	Course introduction, Python foundations, and machine learning overview	How does machine learning work?
Week 03	Regression	
Week 04	Feature engineering	
Week 05	Model selection	
Week 06	Classification	
Week 07	Neural networks	How can I use it to solve engineering problems?
Week 08	Unsupervised learning	
Week 09	Explainable AI, molecular featurization	
Week 10	<i>Midterm presentations</i>	
Week 11	Graph neural networks and molecular property prediction	
Week 12	Convolutional neural networks and computer vision	How can it guide my next decision in research?
Week 13	Gaussian processes and uncertainty quantification	
Week 14	Bayesian optimization and active learning	
Week 15	<i>Thanksgiving Recess</i>	
Week 16	Machine learning best practices, course summary, and advanced topics	

Course Logistics: Management/Discussion

Instructor: *Prof. Jiang* (Swearingen 2C19)

Grader: *Afeez Lawal*

Office hours: TBD

Mid-term: analyze a ML paper (approved by the instructor)

**Final: ML project (topic approved by the instructor):
1 supervised + 1 unsupervised learning**

Format:

	Undergraduates (UG)	Graduate Students
Programming Assignments (4)	40%	40%
Mid-term Presentation	10%	10%
Mid-term Report	10%	10%
Final Project Code and Presentation	30%	15%
Final Project Report	0%	15%
Interactive Engagement	10%	10%
Total	100%	100%

Platforms:

- We are using **Blackboard** for general course management (check here for content, reading, assignments, etc.)
- We will also host a **slack channel**:
For technical/small questions, I encourage Slack. More official correspondence should go through e-mail (SHENGLI@mailbox.sc.edu)

https://join.slack.com/t/eche589fall2026/shared_invite/zt-46myke681-JO8sG5T~R_t69CnC5NdmCw



Blackboard

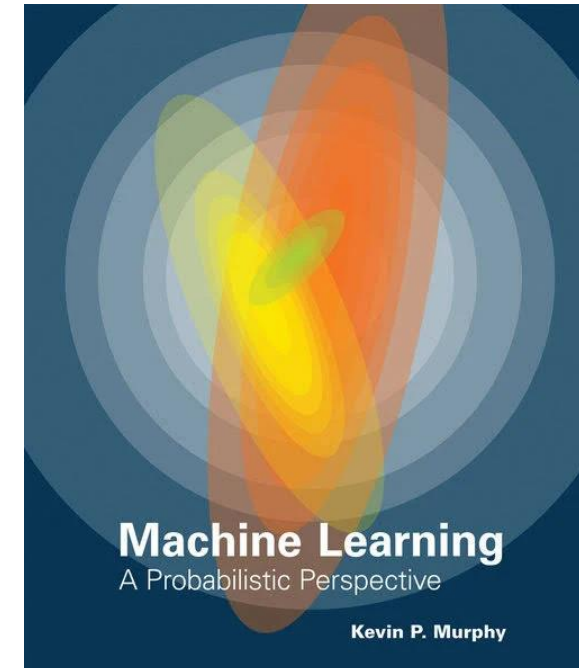
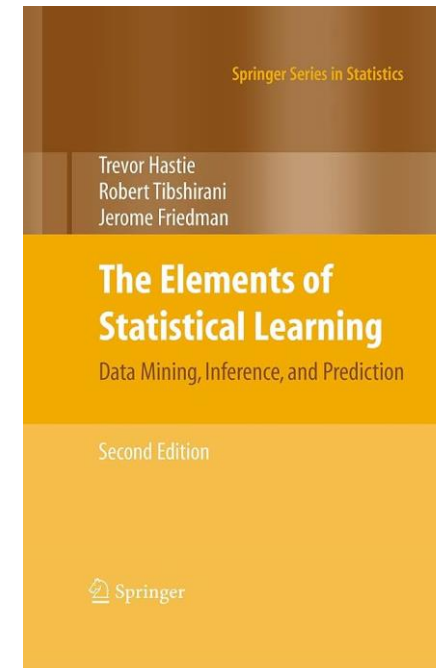
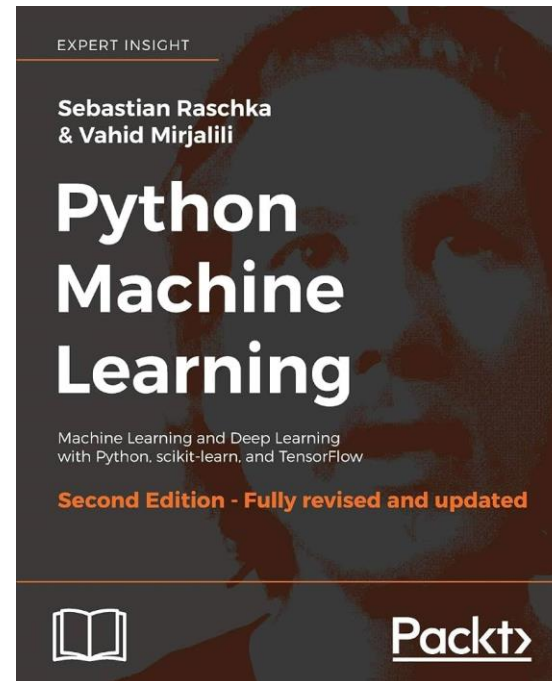
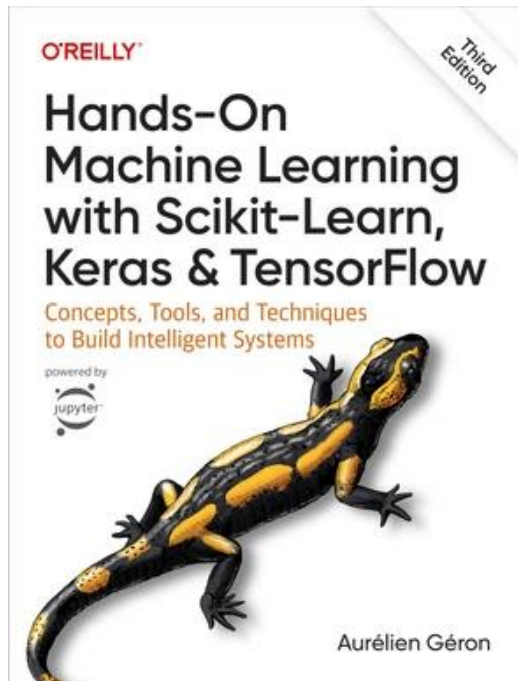


Help and Reference Material

Readings

- **No textbook is required.** Notes, notebooks, and selected readings will be posted in Blackboard. Use official documentation and tutorials when you need more depth. Check examples against the package version used in the course.

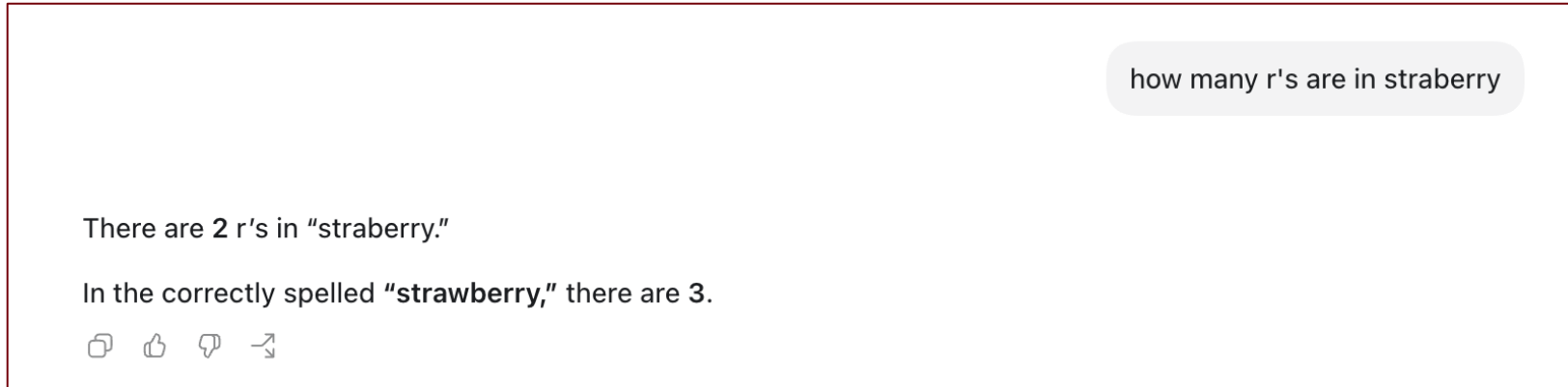
Useful references for independent practice



How We Will Learn Together

- Ask questions early and state uncertainty clearly.
- Test ideas on small examples before trusting a full model.
- Correct errors openly and document what changed.
- Make work reproducible by recording code, data, and software versions.

You will use Python to implement, test, and communicate machine learning workflows.

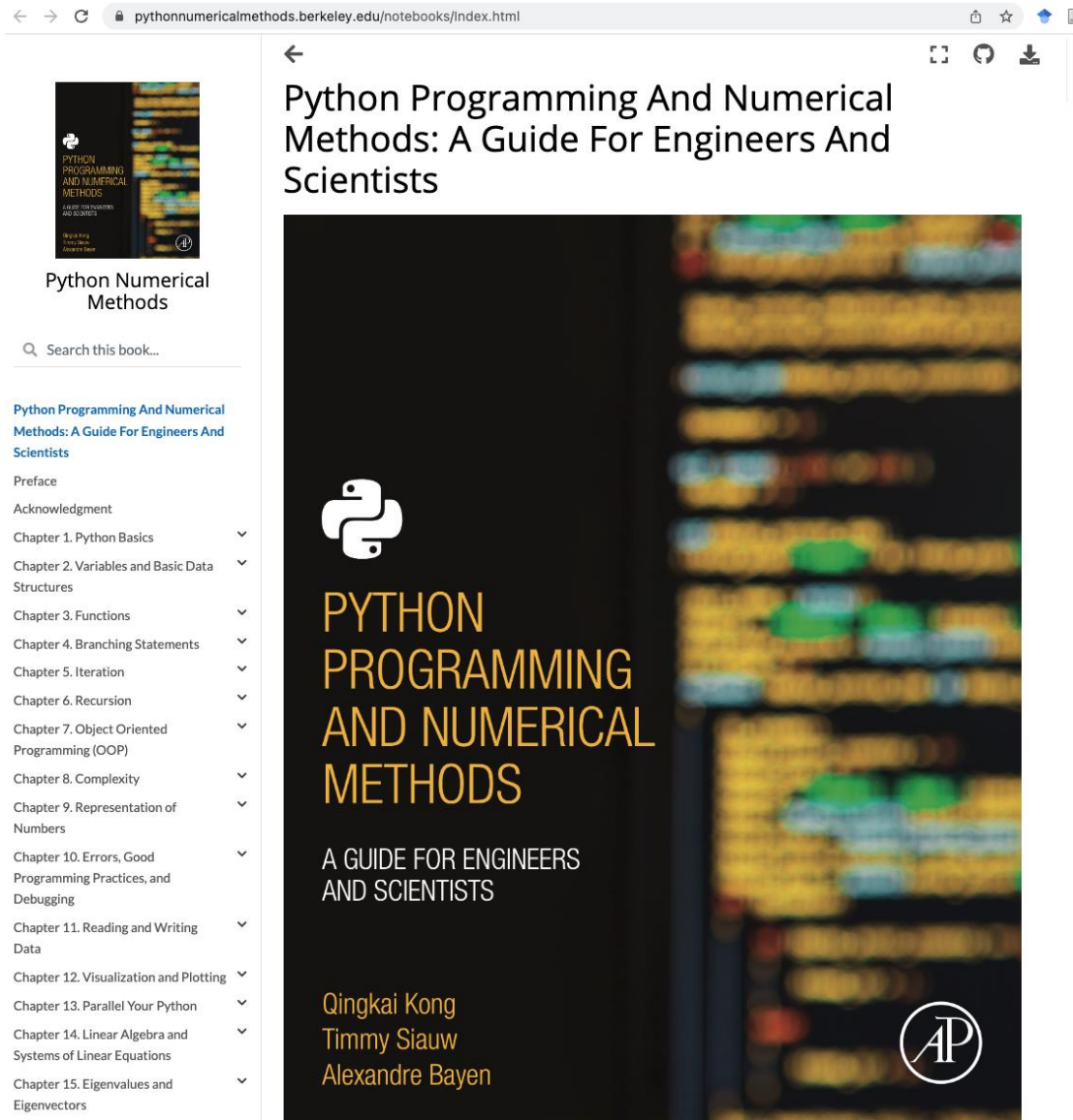


USE OF AI TOOLS

- I **encourage** the use of AI tools to support brainstorming, debugging, code explanation, and language editing.
- However, AI should not replace required analysis, interpretation, original writing, or individual work.
- Disclose the tool, version when available, date, and how it was used.
- You remain responsible for accuracy, citations, privacy, and reproducibility.



Programming in ECHE 589

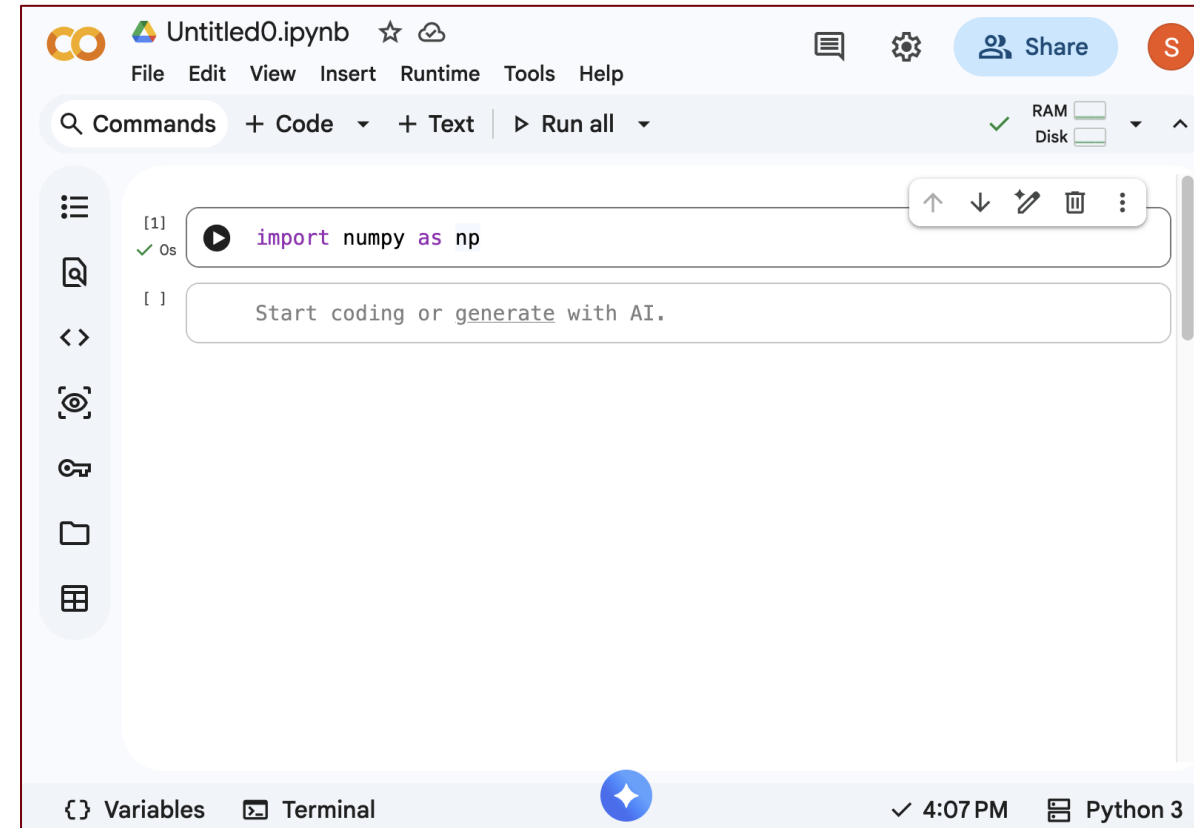


- You have to program in this class, but this is a useful skill to have! Don't be afraid and be engaged.
- There are many resources available online and some that will be provided through the course. These can help you with syntax via mimicry. Understanding may require deeper investigation.
- Use Google, StackOverflow, and **ChatGPT** liberally as needed.
- If you want a companion book to peruse, then this one is freely available online
<https://pythonnumericalmethods.berkeley.edu/notebooks/Index.html>
- Python changes. Don't be surprised if things that once worked no longer do. We have to be adaptive.

Using Google Colab

Using Google Colab, we can obtain working python environments that functionally equip us with everything we need for the course (pretty much... we may need to tweak things here and there)

- Type <https://colab.research.google.com> into your URL.
- Click “New notebook”



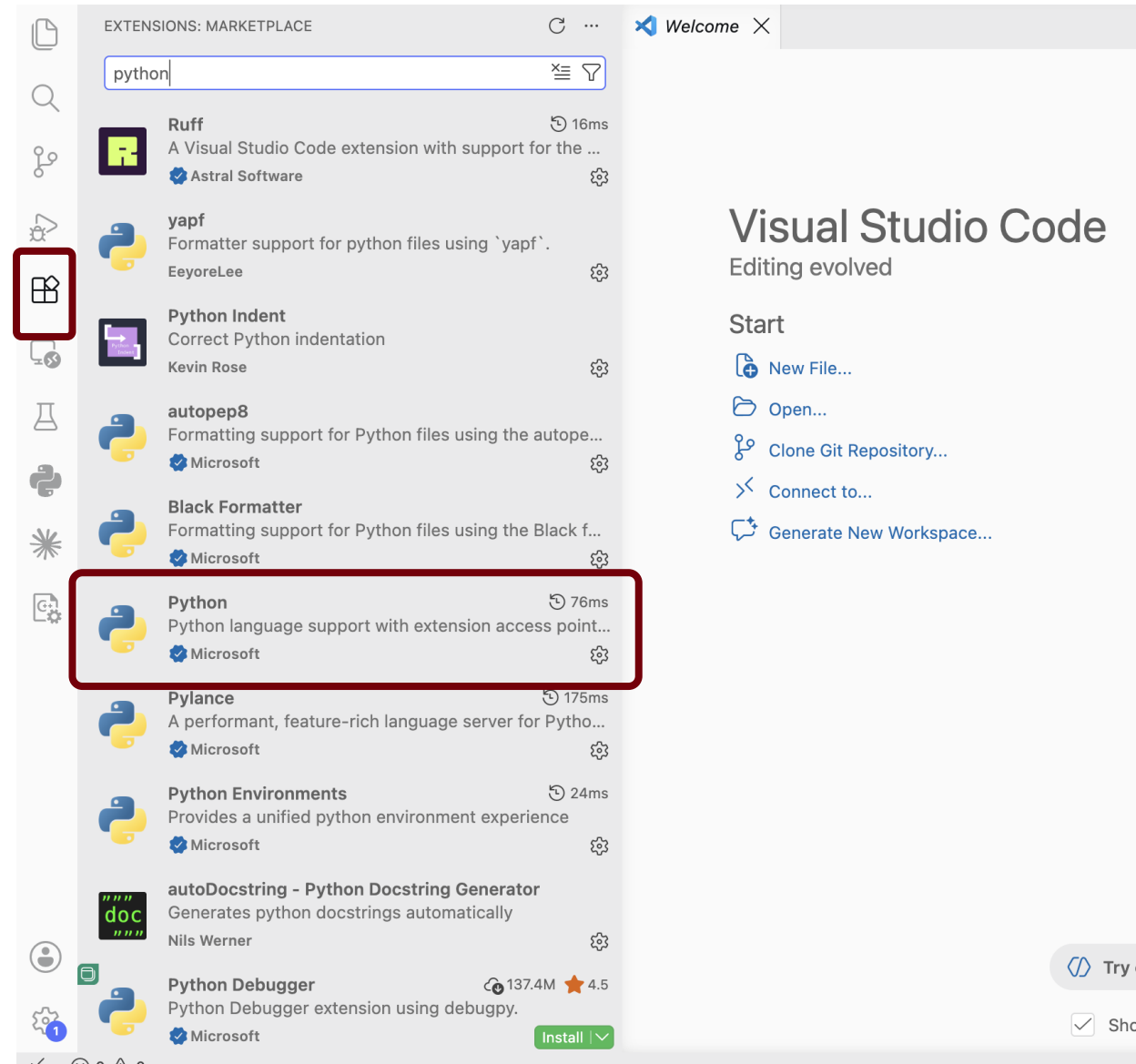
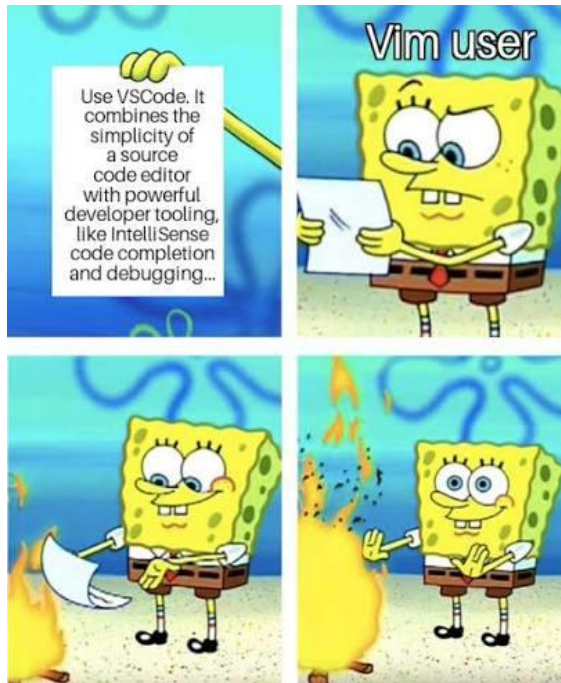
Setting up Python on Your Machine

Step 1 Install VS Code and the official Python and Jupyter extensions.

<https://code.visualstudio.com/>

Other Options:

Anaconda/Spyder,
PyCharm, Sublime Text,
Vim, etc.



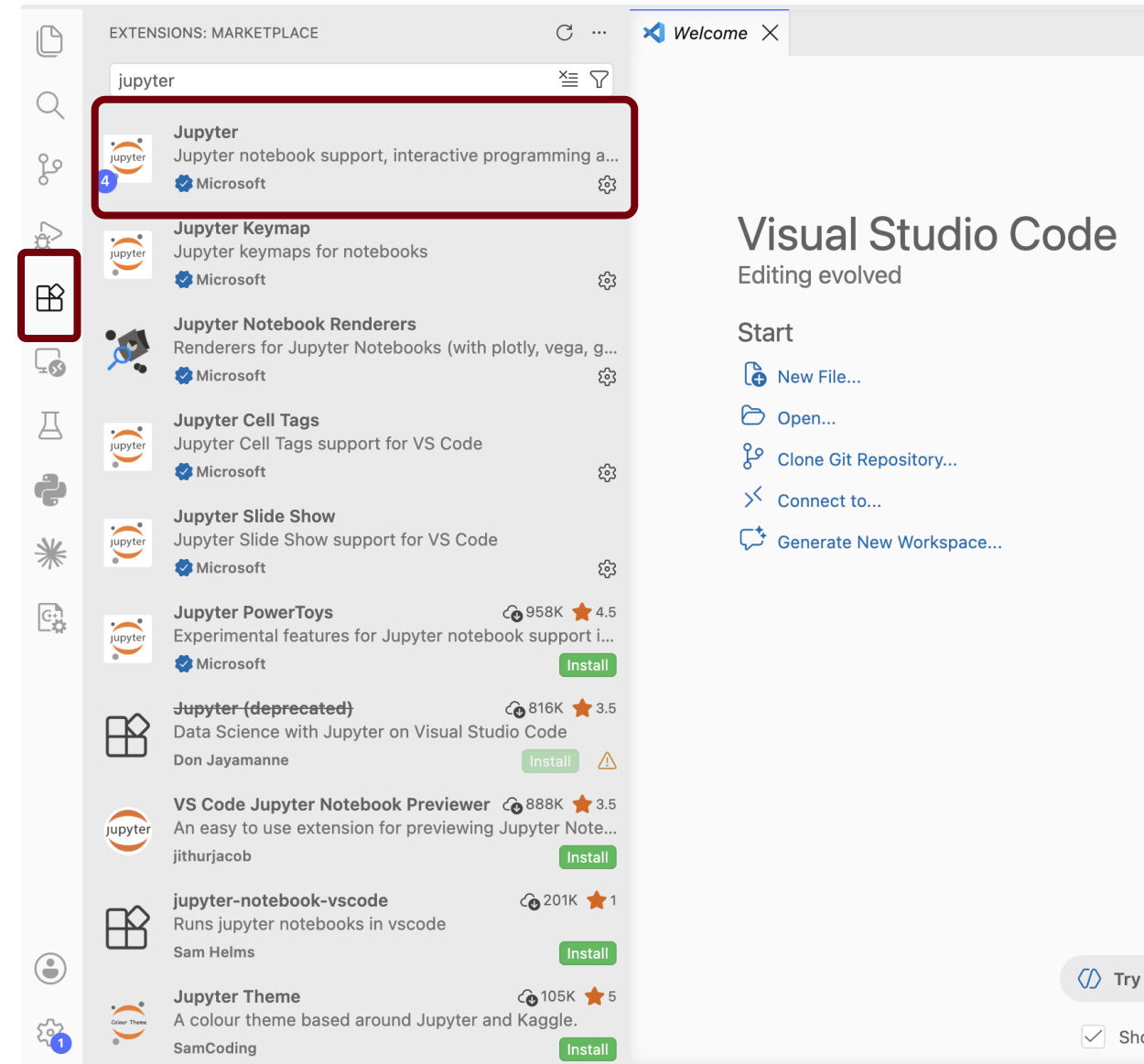
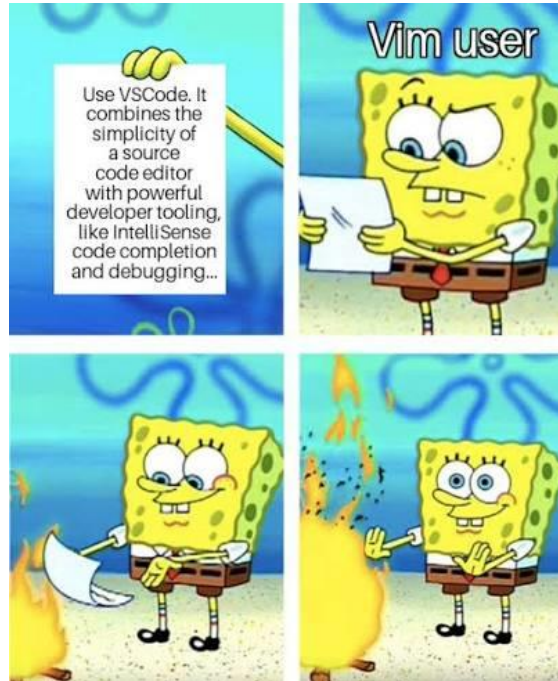
Setting up Python on Your Machine

Step 1 Install VS Code and the official Python and Jupyter extensions.

<https://code.visualstudio.com/>

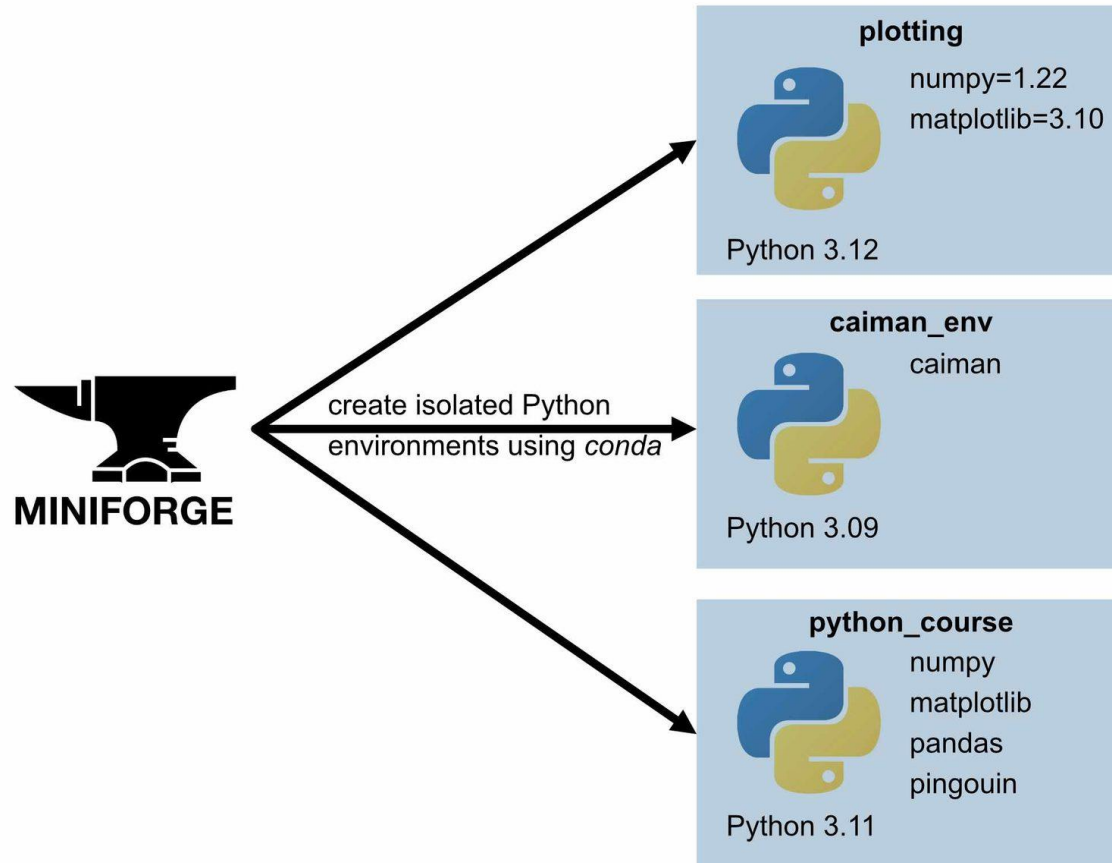
Other Options:

Anaconda/Spyder,
PyCharm, Sublime Text,
Vim, etc.



Setting up a Python Environment on Your Machine

A Python environment is a separate workspace for one project.



- It contains its own Python version.
- Each project can use different versions.
- Changes in one environment do not affect the others.
- We will use **Miniforge** to create and manage these environments.

Step 2 Install Miniforge

<https://conda-forge.org/download/>

Setting up a Python Environment on Your Machine

Miniforge Latest Release (Version 26.3.2-3)



macOS

arm64 (Apple Silicon)



macOS

x86_64 (Intel)



Linux

x86_64 (amd64)



Linux

aarch64 (arm64)



Linux

ppc64le (POWER8/9)



Windows

x86_64

Installation

Basic installation instructions are available below. More detailed instructions are available at conda-forge/miniforge.

Unix-like platforms (Mac OS & Linux)

Download the installer and run `bash Miniforge3-$(uname)-$(uname -m).sh`

Windows

Download and execute the Windows installer.

```
● sj0161@cbe-maw-10 code % pwd
  /Users/sj0161/Documents/03_teaching/usc/eche_589/code
● sj0161@cbe-maw-10 code % cd /Users/sj0161/Documents/03_teaching/usc/eche_589/code
● sj0161@cbe-maw-10 code % ls
  Miniforge3-Darwin-arm64.sh
○ sj0161@cbe-maw-10 code % █
```

Setting up a Python Environment on Your Machine

Step 2 Install Miniforge

```
● sj0161@cbe-maw-10 code % bash Miniforge3-Darwin-arm64.sh
```

```
Welcome to Miniforge3 26.3.2-3
```

```
In order to continue the installation process, please review the license agreement.
```

```
Please, press ENTER to continue
```

```
>>>
```

```
Miniforge installer code uses BSD-3-Clause license as stated below.
```

```
Binary packages that come with it have their own licensing terms and by installing miniforge you agree to the licensing terms of individual packages as well. They include different OSI-approved licenses including the GNU General Public License and can be found in pkgs/<pkg-name>/info/licenses folders.
```

1

```
Do you accept the license terms? [yes|no]
```

```
>>>
```

```
Please answer 'yes' or 'no':
```

2

```
>>> yes
```

```
Miniforge3 will now be installed into this location:  
/Users/sj0161/miniforge3
```

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

```
[/Users/sj0161/miniforge3] >>>
```

```
PREFIX=/Users/sj0161/miniforge3
```

```
Unpacking bootstrapper...
```

```
Unpacking payload...
```

```
Extracting bzip2-1.0.8-hd037594_9.conda
```

```
Extracting c-ares-1.34.6-hc919400_0.conda
```

```
Extracting ca-certificates-2026.5.20-hbd8a1cb_0.conda
```

```
Extracting libcxx-22.1.6-h55c6f16_0.conda
```

3

```
conda config --set auto_activate_base false
```

```
Note: You can undo this later by running `conda init --reverse $SHELL`
```

```
Proceed with initialization? [yes|no]
```

```
[no] >>> yes
```

```
no change      /Users/sj0161/miniforge3/condabin/conda
```

```
no change      /Users/sj0161/miniforge3/bin/conda
```

```
no change      /Users/sj0161/miniforge3/bin/activate
```

```
no change      /Users/sj0161/miniforge3/bin/deactivate
```

```
no change      /Users/sj0161/miniforge3/etc/profile.d/conda.sh
```

```
no change      /Users/sj0161/miniforge3/etc/fish/conf.d/conda.fish
```

```
no change      /Users/sj0161/miniforge3/shell/condabin/Conda.psm1
```

```
no change      /Users/sj0161/miniforge3/shell/condabin/conda-hook.ps1
```

```
no change      /Users/sj0161/miniforge3/lib/python3.13/site-packages/xontrib/conda.xsh
```

```
no change      /Users/sj0161/miniforge3/etc/profile.d/conda.csh
```

```
modified       /Users/sj0161/.zshrc
```

```
==> For changes to take effect, close and re-open your current shell. <==
```

4

Restart VS Code or Terminal

Setting up a Python Environment on Your Machine

```
(base) sj0161@cbe-maw-10 code % mamba --version
2.5.0
(base) sj0161@cbe-maw-10 code % conda --version
conda 26.3.2
(base) sj0161@cbe-maw-10 code % █
```

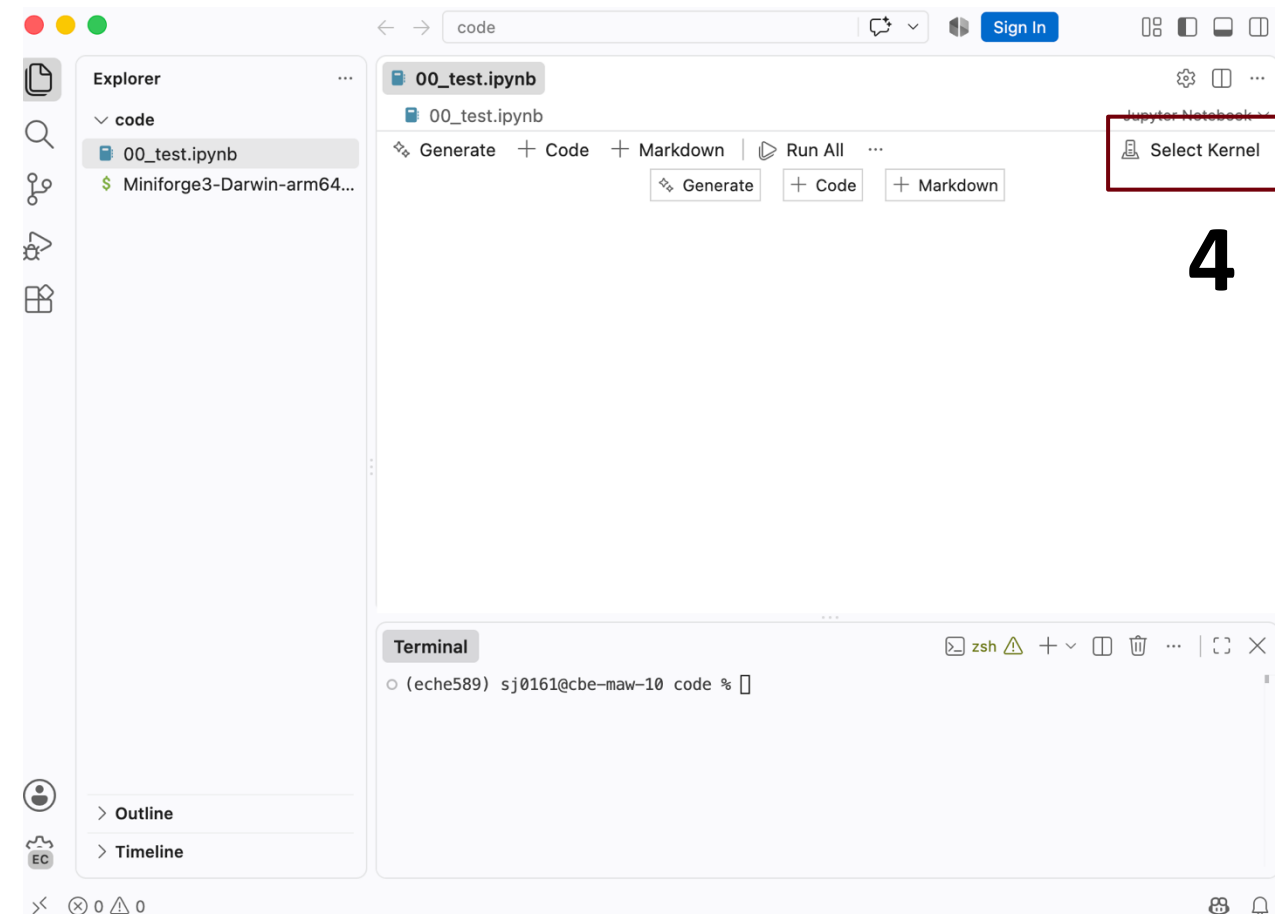
1

```
conda create -n eche589 -c conda-forge \
python \
numpy pandas scipy \
matplotlib \
scikit-learn \
jupyterlab ipykernel ipywidgets \
rdkit shap networkx pillow \
-y
```

2

```
conda activate eche589
```

3



4

The Philosophy of Python

00_test.ipynb > empty cell Jupyter Notebook ▾

Generate + Code + Markdown | Run All Restart ... eche589 (Python 3.14.6)

Generate + Code + Markdown

[1] ✓ 0.0s Python

... The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

Interactive Python Time

`01_python_basics.ipynb`